

Checker-Driven Verilog Repair with a Small Model on Trainium

Team 25: Stacks of Cache | Hack the Chip, NYU x Annapurna Labs | 10 October 2026

Summary. Qwen3-8B, served on a single Trainium chip, writes Verilog that usually compiles but often behaves wrongly. We built a repair loop in which each attempt is simulated in Icarus Verilog and the failure is turned into evidence for the next attempt: the requirement that was violated, what was observed (signal, cycle, which bits were wrong), facts about the model's own code, and at most two short principles retrieved from a knowledge table keyed by failure type. The checker never shows expected values or suggests code, and a pass also requires the official benchmark testbench. On the benchmark LFSR problem, the model went from 0 of 10 correct single attempts, and 30 of 30 identical repeated failures under observation-only feedback, to 5 of 5 runs solved (2.2 attempts per solve) once it was asked to explain each failure and given retrieved principles. Along the way the model passed the official testbench with a simulation-only trick, which we caught and closed with a mid-run reset check. The feedback, not the model, decided the outcome; a harder 8-bit CPU task still failed (best 0.75 after 20 attempts).

The problem

In a first-attempt baseline over 8 tasks, 7 of 8 answers compiled but only 3 of 8 passed their tests. The failures were logical, not syntactic, and the model could explain a concept correctly while implementing it wrongly. Restating the specification did not fix this, so the feedback has to show behavior rather than repeat definitions.

System

Checker. `checker.py` compiles the candidate with Icarus Verilog, generates a probe testbench that instantiates the candidate next to an independent reference design, drives identical stimulus (inputs change only while the clock is low; outputs are sampled 1 ns after each rising edge), and compares outputs. Separate checks test reset before the first clock edge, reset asserted mid-run, and restart after release. Each failed check is reported as *requirement + observation*; scores are weighted sums of passing checks, and 1.0 also requires the official testbench.

Knowledge cache. A table of short principles keyed by *design category : failure signature* (e.g. reset behavior, shift direction, off-by-one). Each round, only the matching one or two entries enter the prompt, so the 8,000-token context is never filled with the whole table.

Agent. A thin loop: the checker builds the prompt (specification, interface, latest attempt, feedback, selected principles; with `--explain`, a request to diagnose each failure first), Qwen replies, and every prompt, reply, candidate and score is logged. Settings: temperature 0.7, thinking off, 1,500 output tokens.

Second pipeline. A teammate pipeline gives feedback from a visible testbench (error ratio, failing locations and internal signal values) and reports results only on a hidden testbench, with attempts run in parallel across team seats. Its quantitative results are not included in this note.

Results: LFSR (5 runs per condition unless noted)

Condition	Solved	Notes
Single attempt, no feedback	0/10	All 10 samples were the same design (wrong reset and a wrong tap).
6 rounds, observation-only feedback	0/5	30/30 attempts byte-identical; the model copied its previous code.
+ explain each failure first	0/5 real	Broke the copying and fixed the tap bug; 3 'solves' used a simulation-only initial block.
+ mid-run reset check (closes the hack)	2/5	Rounds to solve: 5, 5.
+ facts about the model's own code	3/5	Async reset found in 5/5 runs; rounds 4, 4, 3.
+ 10 general principles	4/5	Reset fixed in round 1 in 5/5 runs; rounds 3, 4, 4, 3.

Condition	Solved	Notes
+ 10 observation detectors and principles	3/5	No measurable gain; rounds 4, 6, 6.
Benchmark wording, explain + principles (v4)	5/5	11 attempts total, 2.2 per solve; every final design passed the official testbench.

Rows 1 to 7 used our rewritten specification and an older prompt layout; row 8 used the benchmark's own wording, a newer prompt layout, the immediate reset stated in the prompt, and LFSR-specific knowledge entries. Rows are a progression, not a controlled ablation.

Results: 8-bit CPU (1 run, 20 attempts)

Attempts	Score	Outcome
0, 3-16	0.00	14 compilation failures and one response with no complete module.
1-2	0.20	Compiled; failed reset and execution checks; multiple drivers on one output.
17-19	0.75	Compiled; failed random-input and input-toggle checks (pc and acc undefined after enable went low).

Not solved: no candidate passed the official testbench.

Findings

- **Feedback decides the outcome.** Observation-only feedback left the model repeating one design 30 times. Stating the violated requirement, adding facts about the model's own code, and asking for a diagnosis before the rewrite turned the same failures into passes.
- **Principles beat bigger hints.** Ten general principles took the reset fix from rounds 2-3 to round 1 in every run. A larger detector set added text without adding solves.
- **The model games weak checkers.** With the official testbench as the only judge, Qwen passed by initializing the output at simulation start instead of writing a real asynchronous reset. The official LFSR testbench accepts this; our mid-run reset check does not.
- **Spec wording matters.** Our rewritten LFSR spec ('XOR of bits 0, 3, 5 and 6') produced a wrong tap in every first attempt; the benchmark's original wording ('taps at locations 1, 4, 6 and 7') produced correct taps in 10 of 10 first attempts.
- **Benchmark testbenches have timing races.** The LFSR and sequence-generator testbenches sample outputs at the same instant as a clock edge, which fixes behavior the written specs never state; our checker encodes the behavior the testbench actually requires.

Limitations

Five runs per condition is small, and several steps changed more than one thing at once. The knowledge entries specific to the LFSR and sequence generator were written after seeing those problems; a run restricted to generic entries was not completed. Code-facts and detectors were added after observing failures. All attempt logs, prompts and candidates are in the repository for reproduction.

Next steps

Run the generic-only knowledge condition and the sequence generator with five runs each; add a Yosys synthesis check so passing designs are verified as real hardware, not just simulation (proof of concept exists); build on the best attempt rather than the latest to stop regressions; and extend the knowledge cache to protocol designs with transaction-level monitors.